

The system I developed is based on an event-driven interaction flow that connects three main gameplay elements: coin collection, door unlocking, and chest interaction. The sequence begins when the player collects coins placed throughout the level. Each coin triggers an event when picked up, notifying the game manager that a coin has been collected. This allows the system to keep track of how many coins the player has obtained without tightly coupling the coin objects to other gameplay elements.

Once the required number of coins has been collected, another event is triggered to signal that the condition for opening the door has been met. The door listens for this event and responds by transitioning to an open state, allowing the player to pass through. This creates a clear cause-and-effect relationship: collecting coins directly impacts the environment. After the door is opened, the player can access a chest located beyond it. The chest interaction is either enabled by the door's state or becomes meaningful only after the player reaches it. When the player interacts with the chest, it triggers its own event, which could be used to reward the player, play an animation, or signal progression.

Using events in this system significantly improves modularity. Each object, the coins, the door, and the chest operates independently and does not need direct references to each other. For example, coins do not need to know anything about the door; they only broadcast that they have been collected. Similarly, the door does not need to track coins manually; it simply listens for updates from the event system. This separation makes the code easier to manage, test, and expand. If additional features were added, such as multiple doors or different types of collectibles, they could easily subscribe to the same events without requiring major changes to existing scripts.

Events also improve gameplay responsiveness. Because events are triggered immediately when an action occurs, the system can react in real time. When the player picks up a coin, the game instantly updates the coin count and checks whether the door should open. This creates a smooth and satisfying feedback loop for the player. There is no need for constant polling or checking conditions every frame, which can be inefficient. Instead, the system reacts only when necessary, making it both performant and responsive.

During development, several challenges arose while implementing the event system. One of the main difficulties was correctly setting up event listeners and ensuring that objects were properly connected in the Unity Inspector. At times, events would not trigger as expected because references were missing or incorrectly assigned. This issue was resolved by carefully organizing scripts and double-checking that all required components were attached and linked. Debug logs were also used to confirm whether events were being invoked and received correctly.

Another challenge was ensuring that interactions occurred in the correct order. For example, the chest should not be accessible before the door is opened. Initially, this required additional logic to gate interactions based on game state. This was solved by using boolean flags or conditional

checks tied to event outcomes, ensuring that each step in the sequence only becomes available after the previous one is completed.

Overall, implementing an event-driven system provided a flexible and scalable way to manage gameplay interactions. While there were initial challenges with setup and debugging, the final result is a slightly rough, modular structure that improves both the development process and the player experience.